提高组模拟赛

第 09 套

提高组

题目名称	跳跃	游戏	探测	矩阵
题目类型	传统型	传统型	传统型	传统型
目录	skip	game	detection	matrix
可执行文件名	skip	game	detection	matrix
输入文件名	skip.in	game.in	detection.in	matrix.in
输出文件名	skip.out	game.out	detection.out	matrix.out
每个测试点时限	1 秒	1 秒	1 秒	0.5 秒
内存限制	256 MiB	256 MiB	256 MiB	256 MiB
测试点数目	10	10	10	10
测试点是否等分	是（每点 10 分）	是（每点 10 分）	是（每点 10 分）	是（每点 10 分）

提交源程序文件名

对于 C++ 语言	skip.cpp	game.cpp	detection.cpp	matrix.cpp

编译选项

对于 C++ 语言	`-O2 -std=c++14`

注意事项（请仔细阅读）

1. 文件名及目录名均使用英文小写。
2. 每题程序从指定文件读入，并将结果写入指定文件。
3. 提交源程序文件的名称与英文题名相同。
4. 附加样例位于选手下发包的同名题目目录。
5. 各测试点独立计分。所有题目采用全文比较评测。

跳跃（skip）

【题目描述】

有 N 位同学正在参加集体跳绳，教练是小忍。

每位同学都有一个正整数，表示他们各自的体重基数。正整数越大，表示体重越重，跳绳的频率可能会较低；正整数越小，表示体重越轻，跳绳的频率可能会较高。

小忍会让他们站成一排开始跳绳，但她可以随意调换同学们的左右顺序。但由于某种奇怪的原因，这一整排同学除了最左边的那位同学会按照自己喜欢的频率去跳绳以外，从第二位同学开始，每位同学跳绳的频率都会受到他们左边相邻同学的影响。

对于一名体重基数为 x 的同学来说，如果站在他左边的那位同学体重基数为 y，那么他的跳绳频率将会恰好是他左边那位同学的 $\frac{y}{x}$ 倍。

现在，小忍忽然有了 Q 个疑问，对于第 i 个疑问：

- 是否存在一种顺序，能够使得最右边同学的跳绳频率恰好是最左边同学的 B_i 倍？

【输入格式】

第一行包含两个正整数 N, Q，分别表示总人数以及询问次数。

第二行包含 N 个正整数 $A_1, A_2, \cdots, A_N$，分别表示每位同学的体重基数。

接下来 Q 行，每行一个正整数 B_i，表示询问。

【输出格式】

共 Q 行。对于每个询问，在一行中输出一个字符串，如果存在对应的方案，输出 **YES**，否则输出 **NO**。

【样例 1 输入】

```
5 4
2 6 6 1 3
1
2
3
4
```

【样例 1 输出】

```
YES
YES
YES
NO
```

【样例 1 解释】

$B_i = 1$ 的方案可以是：6 2 1 3 6

$B_i = 2$ 的方案可以是：3 6 1 2 6

$B_i = 3$ 的方案可以是：1 6 2 6 3

【附加样例】

见下发文件中的对应输入及答案。

样例编号	文件
2	**skip2.in**、**skip2.ans**
3	**skip3.in**、**skip3.ans**

【数据范围】

对于 20% 的数据，$N, Q \leq 100$ 。

对于 50% 的数据，$N, Q \leq 2000$ 。

对于 100% 的数据，$2 \leq N \leq 200000$, $1 \leq Q, A_i, B_i \leq 200000$ 。

游戏（game）

【题目描述】

有 N 位同学正在玩惊险刺激的数字游戏，孩子王是小忍。

这 N 位同学自左到右排成一排，每位同学一开始都持有一个整数 A_i。

游戏总共会进行 K 轮，每轮游戏会找出**当前持有的整数最小**的那位同学（如果有多位同学并列最小，则取**最靠左**的一位同学），然后这位同学将会退出游戏，原本与他左右相邻的两位同学（如果有）会变成直接相邻，并且这两位同学（如果有）所持有的整数都会加上刚才退出的这位同学所持有的整数。

问在进行完 K 轮游戏之后，自左到右还未退出的每位同学所持有的整数分别是多少?

【输入格式】

第一行包含两个整数 N, K，表示总人数以及游戏进行的轮数。

第二行包含 N 个整数 $A_1, A_2, A_3, \cdots, A_N$，分别表示一开始自左到右每位同学所持有的整数。

【输出格式】

输出 $N - K$ 个整数，整数间以单个空格隔开，代表 K 次游戏后自左到右还未退出的每位同学所持有的整数。

【样例 1 输入】

```
4 3
1 2 1 2
```

【样例 1 输出】

```
7
```

【样例 1 解释】

每轮游戏的结果如下:

- [1] 2 1 2
- 3 [1] 2
- 4 [3]
- 7

【样例 2 输入】

```
5 3
4 2 5 1 3
```

【样例 2 输出】

```
6 12
```

【样例 2 解释】

每轮游戏的结果如下:

- 4 2 5 [1] 3
- 4 [2] 6 4
- 6 8 [4]
- 6 12

【样例 3 输入】

```
10 6
5 4 6 8 2 1 7 4 9 3
```

【样例 3 输出】

```
19 11 15 16
```

【附加样例】

见下发文件中的对应输入及答案。

样例编号	文件
4	**game4.in**、**game4.ans**

【数据范围】

对于 20% 的数据，$N \leq 10000$

对于 100% 的数据，$1 \leq K < N \leq 500000$, $0 \leq A_i \leq 10^8$

探测（detection）

【题目描述】

有一条高速公路，它可以简化成一条数轴。一开始所有的 N 辆车都在数轴的负半轴，第 i 辆车位于数轴 A_i 处，它们都朝着正半轴方向行驶，并且速度都是每秒 1 个单位长度。

共有 M 种规格的车辆探测器，每种车辆探测器都有一个属性 R_i，表示其探测半径。如果一个探测半径为 R_i 的车辆探测器被放置在数轴上的位置 X 处，那么它将能够探测到 $[X-R_i, X+R_i]$ 范围内的车辆。只要探测范围内有至少一辆车存在，探测器的提示灯就会亮起，否则提示灯将会是熄灭状态。

提示灯的功率是 P 瓦，也就是说在提示灯亮起的时候，每 1 秒都需要消耗 P 焦耳的电量。另外，提示灯从熄灭状态变成亮起状态需要消耗额外的电量，每次消耗 C 焦耳。从亮起状态转为熄灭状态不会消耗额外电量。当然，如果提示灯原本已经是亮起状态，并且此时又有另一辆车进入了该探测器的探测范围，这将**不会**产生额外的电量损失。

由于车辆探测器用的提示灯都是同一个品牌统一规格的，所以 P 和 C 是固定的常数。

已知车辆探测器使用了十分先进的科技，运行过程中其他部分几乎不会耗电，主要电量的消耗均来自于提示灯。

现在，对于探测范围为 R_i 的探测器，我们会将其放置在数轴上的 R_i 位置处，使其能够探测到 $[0, 2R_i]$ 范围内的车辆。问在当前车辆的行驶状况下，每种探测器分别会消耗多少焦耳的电量?

【输入格式】

第一行包含四个整数 N, M, P, C，分别表示车的数量、车辆探测器的种类数，以及提示灯的两个属性。

第二行有 N 个整数 $A_1, A_2, \cdots, A_N$，第 i 个整数 A_i 表示第 i 辆车在数轴上的位置，保证给定序列是递增的。

第三行有 M 个整数 $R_1, R_2, \cdots, R_M$，第 i 个整数 R_i 表示第 i 种车辆探测器的探测半径。

【输出格式】

输出 M 行，第 i 行表示第 i 种车辆探测器会消耗的总电量。

【样例 1 输入】

```
3 3 2 6
-8 -5 -1
1 2 3
```

【样例 1 输出】

```
30
28
32
```

【样例 1 解释】

对于第二种车辆探测器：

- $t = 0$，初始情况；
- $t = 1$，第一辆车进入探测器范围，消耗额外的启动电量 C；
- $t = 5$，第二辆车进入探测器范围，第一辆车离开探测器范围，此时灯不会灭，因此不会有额外 C 的消耗；
- $t = 8$，第三辆车进入探测器范围；
- $t = 12$，所有车离开探测器范围。

从 $t = 1$ 开始到 $t = 12$，灯一直亮着，所以电量消耗是 $11 \times P + 1 \times C = 11 \times 2 + 1 \times 6 = 28$。

【附加样例】

见下发文件中的对应输入及答案。

样例编号	文件
2	**detection2.in**、**detection2.ans**
3	**detection3.in**、**detection3.ans**

【数据范围】

对于 30% 的数据，$N, M \leq 20$ 。

对于 100% 的数据，$1 \leq N, M \leq 10^5$, $1 \leq P, C \leq 10^6$, $-2 \times 10^6 \leq A_i < 0$, $\forall_{i \in [2,N]} : A_{i-1} < A_i$, $1 \leq R_i \leq 10^8$ 。

矩阵（matrix）

【题目描述】

现有一个 N 行 N 列的 $0/1$ 矩阵。

共有以下三种操作：

1. 选择某一行，将整行所有数字进行 $0/1$ 翻转（即 $0 \to 1$，$1 \to 0$）。
2. 选择某一列，将整列所有数字进行 $0/1$ 翻转。
3. 选择矩阵内某个指定位置，将该位置的数字进行 $0/1$ 翻转。

问是否存在一种方案，使得在最多进行 K 次操作 3 的前提下，能够将整个矩阵变为全 0?

【输入格式】

第一行包含一个正整数 T，表示测试数据组数。

每组数据格式如下：

第一行包含两个整数 N, K，分别表示矩阵大小以及操作 3 的次数限制。

接下来 N 行，每行包含 N 个字符，仅由 **0** 或 **1** 组成，表示给定的矩阵。

【输出格式】

对于每组数据，如果存在合法方案，在一行内输出 **YES**，否则输出 **NO**。

【样例 1 输入】

```
3
2 0
01
01
3 1
100
001
000
4 2
1001
0010
1011
1001
```

【样例 1 输出】

```
YES
NO
YES
```

【样例 1 解释】

【对于第三组测试数据】

```
1001      0001      0000      0000
0010  ->  1010  ->  1011  ->  0100
1011      0011      0010      0010
1001      0001      0000      0000
```

然后将剩余的两个 **1** 借助操作 3 改为 **0** 即可。

【附加样例】

见下发文件中的对应输入及答案。

样例编号	文件
2	**matrix2.in**、**matrix2.ans**
3	**matrix3.in**、**matrix3.ans**

【数据范围】

对于 20% 的数据，$K = 0$ 。

对于另外 20% 的数据，$N \leq 100$ 。

对于 100% 的数据，$1 \leq T \leq 5$, $1 \leq N \leq 1000$, $0 \leq K \leq N$ 。

请注意本题特殊的时间限制。